KEY — Biweekly Assessment BW06 — Weeks 11–12 (Trainer only)

Do not share with students. Excluded from student pack (`*_KEY.md`).

A. MCQ answers

1. **a** — Points at one PRIMARY KEY value in `members` so a loan belongs to one member
2. **b** — Only rows where the JOIN conditions match (members who have a loan to a book)
3. **c** — `LEFT JOIN` stalls to orders (unmatched order columns are NULL)
4. **b** — Undoes uncommitted work since the transaction started
5. **a** — Speeds `WHERE` / `JOIN` / `ORDER BY` on `author`; writes pay a small extra cost
6. **a** — Auto-closes the `Connection` (`AutoCloseable`) even if `SQLException` is thrown
7. **b** — `ps.setString(1, author)` — JDBC `?` indexes are **1-based**
8. **c** — `executeUpdate` (returns the row count)
9. **a** — A typed title can change the command (SQL injection), e.g. `' OR '1'='1`
10. **b** — Env (`MYSQL_PASSWORD`) or a local `.env` (commit `.env.example` only)
11. **c** — `<main>`
12. **a** — Use `<label for="...">` matching each input `id`, and `method="post"` so payment data is not stuck in the URL

B. Coding — model points

1. `PreparedStatement` `?` only, try-with-resources, password from env (`.env` reader also OK). `listByStall` uses `executeQuery`; `addItem` uses `executeUpdate`. No `Statement`, no `"'" + stall + "'"`, no password literal, no Spring / JPA / Lombok / `DataSource` / `application.yml`.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class CanteenStock {
    static final String URL = "jdbc:mysql://localhost:3306/training_db"
            + "?allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=Asia/Kolkata";

    public static void main(String[] args) {
        String password = System.getenv("MYSQL_PASSWORD");
        if (password == null || password.length() == 0) {
            System.out.println("Set MYSQL_PASSWORD first");
            return;
        }
        try (Connection conn = DriverManager.getConnection(URL, "root", password)) {
            listByStall(conn, "South");
            addItem(conn, "Filter coffee", 25, "South");
            listByStall(conn, "South");
        } catch (SQLException e) {
            System.out.println("Could not talk to MySQL: " + e.getMessage());
        }
```

```
    }

    static void listByStall(Connection conn, String stall) throws SQLException {
        String sql = "SELECT name, price_inr FROM canteen_items WHERE stall = ? ORDER BY name";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, stall);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getString("name") + " ₹" + rs.getInt("price_inr"));
                }
            }
        }
    }

    static void addItem(Connection conn, String name, int priceInr, String stall)
            throws SQLException {
        String sql = "INSERT INTO canteen_items (name, price_inr, stall) VALUES (?, ?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, name);
            ps.setInt(2, priceInr);
            ps.setString(3, stall);
            System.out.println("Inserted rows: " + ps.executeUpdate());
        }
    }
}
```

If `MYSQL_PASSWORD` is unset: `Set MYSQL_PASSWORD first`. With MySQL + table + Connector/J on `-cp`, expect existing South rows, then `Inserted rows: 1`, then the list including `Filter coffee ₹25`. Award marks for `?` + try-with-resources + env password + `SQLException`. A `.env` helper (kit 16 style) is fine. Hard-coded password or `Statement` concat is a miss even if the SQL "works" on happy input.

2. Semantic landmarks, POST, `label for` / `id` / `name`, `fieldset` + `legend` radios, `<button type="submit">`. No CSS / JS. Wrapping labels (`<label>Name <input …></label>`) is acceptable if the control is still named.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CampusCart / TechMart Canteen Order</title>
</head>
<body>
  <header>
    <h1>CampusCart / TechMart Canteen</h1>
  </header>
  <main>
    <form action="/canteen/order" method="post">
      <label for="student">Student name</label>
      <input id="student" name="student" type="text" required>

      <label for="item">Item</label>
      <input id="item" name="item" type="text" required>

      <label for="qty">Quantity</label>
      <input id="qty" name="qty" type="number" min="1" required>

      <fieldset>
        <legend>Pay with</legend>
        <label><input type="radio" name="pay" value="upi" checked> UPI</label>
        <label><input type="radio" name="pay" value="cash"> Cash</label>
      </fieldset>
```

```
      <button type="submit">Place order</button>
    </form>
  </main>
  <footer>
    <p>Prices in ₹ · KSR Datavizon campus</p>
  </footer>
</body>
</html>
```

Award marks for POST + matching labels + one a11y control (`fieldset` / `legend`, or `aria-label` on `nav`, or meaningful `alt`). Flex / media queries / `fetch` are out of window — do not require them; deduct if the page is CSS-layout-only with unlabeled inputs.

C. Short answer — model

INNER JOIN `members` to `loans` ON `members.member_id = loans.member_id` keeps only members who have at least one loan — Priya with no loan **disappears**. LEFT JOIN from `members` to `loans` keeps every member; loan columns are NULL when there is no match (the “stalls with no orders” idea). A third table `books` is the same pattern: join `loans.book_id` to `books.book_id`. In JDBC, that SQL still uses `WHERE m.member_id = ?` (or bind the search value) — never `"'" + memberId + "'"`. Spring / JPA / CSS are not the point of this item.

Suggested scoring

Section	Marks
A MCQ (1 each)	12
B Coding (6 each)	12
C Short	6
Total	**30**